

Tools: Random Numbers, Monte Carlo, Intro to Estimators

Dr Lily Asquith (Lily, she/her)



Tools

- Random/Pseudorandom Numbers
- Monte Carlo
- Requirements for an Estimator
- Sample Mean and Variance as Estimators
- Likelihood and Bayes recap
- Maximum Likelihood, Least Squares, Residuals, Chi Squared

Intro, reminder of big picture

Estimating parameters and their uncertainties has been our goal, and we are now pretty much ready to do that.

We have used **PseudoRandom Number Generators** many times already, every time we pull a set of RVs from a `scipy.stats` pdf for example. We will give a little nod to where they come from.

Monte Carlo methods are important for simulating fake data and for estimating parameters.

We will see that **Parameter Estimation** comes down to maximising the Likelihood function, or equivalently to minimising the residuals (Chi Squared is the sum of the squared residuals).

Next week we will go through Maximum Likelihood Estimation in full. The process is much more intuitive and friendly than some descriptions would have us believe. There are a few bits of math, but we will be grand I think.

Random Numbers



Credit: DILBERT © 2001 Scott Adams [All rights reserved].

Random Numbers

A Random Number is a term used to describe a value that is produced by a random (unpredictable) process.

There are many things in the physical world that exhibit random behaviour (such as quantum mechanical processes), but the instruments we use to measure the behaviour often destroy/hide the randomness.

We can extract (a limited number of) truly random numbers from eg random.org, which uses atmospheric noise.

True Random Number Generation (TRNG) is an active area of research, and is rapidly changing. Whatever I write here will probably be out of date in two years. A recent exciting development was [published in Nature](#) this summer.

PseudoRandom Numbers (PRNs)

A Pseudorandom Number is a term used to describe a number in a sequence that appears random, but is produced by a deterministic* process.

Note that when we draw RVs from a distribution using python libraries, we are using their PseudoRandom Number Generators (PRNGs).

```
from numpy import random
# It is called random, but should be called pseudorandom
crvs_x = random.uniform(0,100,10)
```

Human beings tend to have fixed ideas of what random should look like, which is amusing if you think about it. If you truly had a random shuffle on your music playlist, you would get many repetitions. People don't like that.

*Deterministic: an outcome is caused by preceding events.

An Outdated PRNG

A Multiplicative Linear Congruential* Generator (MLCG) is an algorithm that produces sequences of pseudorandom numbers using the rule:

$$n_{i+1} = a n_i \bmod(m)$$

The parameters a , m and n_0 (n_0 is the **Seed**) must be chosen with care to ensure the periodicity does not cause problems. Check out the terrible example of choices below where the produced sequence repeats after just 6 numbers.

```
# DEMO of MLCG PERIODICITY - repeats after six numbers!#
import numpy as np
def MLCG(a, m, seed, size):
    sequence = np.zeros(size)
    sequence[0] = seed
    for i in range(0, size-1):
        sequence[i+1] = (a * sequence[i]) % m
    return sequence
print( MLCG(a=3, m=7, seed=1, size=10) ) #[1. 3. 2. 6. 4. 5. 1. 3. 2. 6.]
```

Congruence: the quality or state of agreeing, coinciding.

Linear Congruential Generators

A modification to improve the MLCG behaviour is to use an increment c such that the Linear Congruential Generator (LCG) is:

$$n_{i+1} = (a n_i + c) \bmod(m)$$

It is still crucial to choose the parameters with care. The C programming language used $a = 1,664,525$ based on a bunch of research into behaviour of numbers.

The modulus m should be a prime or power of 2 (for efficiency of computation) and should be large enough to ensure the minimum level of periodicity for whatever you are doing.

Note that the PRNs supplied by eg python libraries are state-of-the-art and it is very unlikely you will want to employ your own methods!

The Seed

The Seed is a starting value that entirely controls the sequence of pseudorandom numbers produced.

If we use the same seed in a PRNG, we will get the same sequence. This is very useful for testing.

With python we have to set the seed to the same value **every time we call rand** (for example), otherwise the sequence will use a different seed (note that this is not thread-safe).

If you do not set the seed, numpy/scipy will set it using nondeterministic data from your computer operating system.

The Seed

```
# DEMO of SEED #  
import numpy as np  
from numpy import random  
np.random.seed(0) # set the seed  
np.random.rand(3) # generate 3 PRNs  
#array([0.5488135 , 0.71518937, 0.60276338])  
np.random.seed(0) # set the seed  
np.random.rand(3) # generate 3 PRNs  
#array([0.5488135 , 0.71518937, 0.60276338])  
np.random.rand(3) # generate 3 PRNs without setting the seed  
#array([0.54488318, 0.4236548 , 0.64589411])
```

We cannot just set the seed once and then call the generator over and over again and expect the same sequence.

The Mersenne Twister & PCG

The **Mersenne Twister** is an algorithm used by python and most other programming languages to generate pseudorandom numbers.

It is named for its use of the Mersenne prime, $2^{19937} - 1$, as the period length. It is called a Twister because it's a cool name and because it is a 'twisted' version of an earlier algorithm (GFSR).

PCG64 (Permuted Congruential Generator, 64-bit) is the PRNG used by numpy. I tested it, and it is faster than the scipy default Mersenne Twister!

For those interested in the algorithms, see the [1997 Mersenne Twister paper](#) from Matsumoto & Nishimura and the [2014 PCG paper](#) from O'Neill.

PCG (numpy) versus Mersenne Twister (scipy)

Note: to do this test properly you would want to do it many times and take the average.

```
import timeit
N=1_000
glob = {'N': N}
t_sp = timeit.timeit(setup='import scipy as sp;
sp_init = sp.stats.norm()',
stmt='sp_init.rvs(size=N)',globals=glob )
print("Scipy MT time:{} for {} norm PRNS.".format( t_sp, N ) )

t_np = timeit.timeit(setup='import numpy as np;
np_init= np.random.default_rng()',
stmt='np_init.normal(size=N)',globals=glob )
print("Numpy PCG64 time:{} for {} norm PRNS.".format( t_np, N ) )

> Scipy MT time:27.727043333999998 for 1000 norm PRNS.
> Numpy PCG64 time:5.511924125000004 for 1000 norm PRNS.
```

The Monte Carlo Method

Monte Carlo is a code name, because during WW2 everything had to have a stupid name, because spies.

Stanislaw Ulam was playing solitaire, and wanted to know the probability of it coming out successfully. He wondered if the best way to determine this wouldn't be to just play the game a very large number of times and observe the frequency of success.

Ulam's 'brute force sampling' idea coincided (not a random coincidence) with the invention of computers, so it turned out to be a very good one.

Ulam apparently stated that it would do okay 'until the mathematicians arrive'. It seems like they have not yet arrived.

The Monte Carlo Method

In its simplest form, the Monte Carlo method involves 'rolling a dice' (aka using a PRNG) to create random data points that can be used for estimating something.

In its more general form, the Monte Carlo Method involves a bit more than just dice rolling. Sequences of PRNs can be generated using an algorithm such as the Mersenne Twister, and then may need to undergo some **Transformation** such that they produce whatever distribution is of interest to the user. The values extracted from the transform can be used to estimate some property of the 'fake data' produced.

Python methods produce PRNs drawn from a large choice of distributions, so the Transform part of the MC Method is less relevant than it was back in the day when we could only produce basic (uniform) PRN sequences.

Transformation of PRN Sequences

The **Transformation Method** converts a uniform (flat) distribution of random numbers U into a shapely distribution of choice X by determining the function $X(U)$. This is done by integrating the PDF and setting it to U , then solving for X .

The **Acceptance/Rejection Method** is a 'hit and miss', method that can be used when the integral is too tricky. It involves drawing a box to contain our distribution, and then creating a histogram (more boxes) to define acceptable areas for our uniform numbers to fall in. This method can be very inefficient, but can be improved by drawing something a little more complicated than a single box (like , two boxes...)

The **Weighted Sampling Method** uses all of the uniform PRNs, and stores a weight corresponding to the value of the (desired) PDF at that point. Then, whenever the PRN is used (to fill histograms etc), we can weight the event accordingly.

Example: Transformation Method

Uniformly distributed sequence: $u = \{u_1, u_2, \dots, u_n\}$

We want them to be $x \sim \text{Expon}(\lambda)$: $f_X(x; \lambda) = \lambda e^{-\lambda x}$

$$u = \int_0^x \lambda e^{-\lambda x} dx = -\frac{1}{\lambda} \lambda e^{-\lambda x} \Big|_0^x = -1[e^{-\lambda x} - e^{-\lambda 0}] = 1 - e^{-\lambda x}$$

$$1 - u = e^{-\lambda x}$$

$$\ln(1 - u) = -\lambda x$$

$$-\frac{1}{\lambda} \ln(1 - u) = x(u) : \text{ we have an exponentially distributed sequence, } x$$

I am sure you can imagine, this is a very simple example of something that can be very complex!

Point Estimators

A True Parameter (or set of parameters) is θ . We estimate the value of this parameter using a dataset of measurements x_{obs} .

A **Point Estimator** is a function we use to make the Estimate: $\hat{\theta}(x_{obs})$

The value returned by the function is the Point Estimate $\hat{\theta}$ of the true parameter θ .

We use 'hat notation' to indicate an Estimate (value, or Statistic) and to indicate an Estimator (function, or Statistic!):

$$\hat{\theta} = \hat{\theta}(x_{obs})$$

Side note on Statistics

A **Statistic** is a property of the data. Examples include:

The **sample** mean: $\frac{1}{N} \sum_i^N x_i$ (also the mode, median)

The **sample** variance: $\frac{1}{N} \sum_i^N (x_i - \bar{x})^2$ (also the standard deviation).

The minimum value of a **dataset**: $\min(x)$ (also the maximum, also any function of these).

The most likely value of a parameter, **given the data**: $\hat{\theta} = \operatorname{argmax}(\ln \mathcal{L}_{\theta})$.

The Expected Value $\mu = E[X]$ and True Variance $V[X]$ are not statistics, as they do not rely on the data.

Interval Estimator

An **Interval Estimator** for a parameter is also a function $\hat{\theta}(x_{obs})$ computed on a dataset x_{obs} .

Rather than returning a single value (as per a point estimator), it returns:

- 1 A range (upper and lower boundary): the Confidence Interval
- 2 A Confidence Level

For example, an Interval Estimator might return a Confidence Interval of (3.20, 3.25) and a Confidence Level of 99%*

* Recall the meaning of this is that in 99% of repetitions of the experiment, we would expect to find a value between (3.20, 3.25)

Requirements for an Estimator

An Estimator is constructed with the aim of being Consistent, Unbiased, and having Minimum Variance. In English: we want our Estimator to produce a good estimate.

Consistent: Estimate should approach true value as $N \rightarrow \infty$.

$$\hat{\theta} \rightarrow \theta \text{ as } N \rightarrow \infty$$

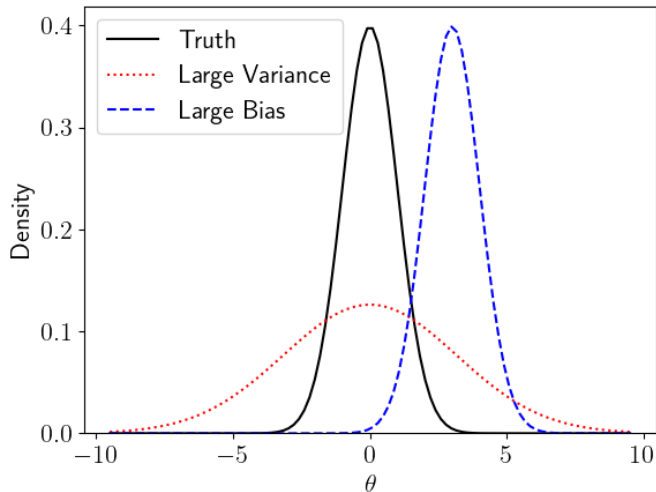
Unbiased: The difference between estimate and the true parameter should be small, preferably zero.

$$b = E[\hat{\theta}] - \theta$$

Minimum Variance: We don't want too much spread in the estimated values (we want *consistency*, but we used that word already).

$$V[\hat{\theta}] = E[\hat{\theta}^2] - E^2[\hat{\theta}] \equiv E[(\hat{\theta} - \mu)^2]$$

Bias and Variance: Visually



The Mean and Variance of a Sample

The **formulae for calculating** the mean and variance of a sample are statistics. They are **Estimators** for the true mean and variance of the Underlying Distribution. The **values they return** are the **Estimates**.

The Sample Mean: $\bar{x} = \frac{1}{N} \sum_i^N x_i$ is an **Estimate**, $\hat{\mu}$ for the true mean μ .

The Sample Variance: $V[x_N] = \frac{1}{N} \sum_i^N (x_i - \bar{x})^2$ is an **Estimate**, $\hat{V}[X]$ for the true variance $V[X]$.

We will check their Consistency & Bias. (Variance checks can be long, but they are easy to find online/ in text books if you want to practice some proofs.)

Sample Mean: Estimator Consistency

The Sample Mean: $\bar{x} = \frac{1}{N} \sum_i^N x_i$ is an Estimate, $\hat{\mu}$ for the true mean μ .

Consistency Check: Does $\hat{\mu} \rightarrow \mu$ as $N \rightarrow \infty$?

Recall the Weak LLN: $\lim_{N \rightarrow \infty} P(|\bar{x} - \mu| \geq \alpha) = 0$ for all $\alpha > 0$

Translation: If every possible measurement of x is included in the calculation of $\bar{x}$, the probability of there being any measurable difference between $\bar{x}$ and $\hat{\mu}$ is zero.

The Sample Mean is a Consistent Estimator for the True Mean.



Sample Mean: Estimator Bias

The Sample Mean: $\bar{x} = \frac{1}{N} \sum_i^N x_i$ is an Estimate, $\hat{\mu}$ for the true mean μ .

Bias Check: $b = E[\hat{\mu}] - \mu$

We showed (last week) that $E[\bar{x}] = E[X]$

Translation: the expected value of the mean is the 'mean of means': if we calculate the means of all possible datasets measuring a RV, the mean of those means will be the true mean.

In estimator notation this is: $E[\hat{\mu}] = \mu$: the bias is zero.

The Sample Mean is an Unbiased Estimator for the True Mean. ✓

The Sample Variance

The Sample Variance: $V[x_N] = \frac{1}{N} \sum_i^N (x_i - \bar{x})^2$ is an **Estimate**, $\hat{V}[X]$ for the true variance $V[X]$.

To check the Consistency, Bias, and Variance of the **Estimator**, we need to write down $\hat{V}[X] = V[x_N]$ in terms of the True Variance, $V[X]$.

This is not a lecture proof (too long!), but as the proof is not included in the notes or textbook for this module (or the textbook I have in my office!), I have written it out for you on the next slide.

The conclusion is

$$\hat{V}[X] = V[x_N] = \frac{N-1}{N} V[X]$$

$$V_x = \frac{1}{N} \sum_i (x_i - \bar{x})^2$$

$$E[V_x] = E\left[\frac{1}{N} \sum_i (x_i - \bar{x})^2\right]$$

$$= \frac{1}{N} E\left[\sum_i (x_i - \bar{x})^2\right]$$

$$= \frac{1}{N} \sum_i E[(x_i - \bar{x})^2]$$

$$= \frac{1}{N} \sum_i E[(x_i - \mu - (\bar{x} - \mu))^2]$$

$$= \frac{1}{N} \sum_i E[(x_i - \mu)^2 + (\bar{x} - \mu)^2 - 2(x_i - \mu)(\bar{x} - \mu)]$$

$$= \frac{1}{N} \sum_i \left[E[(x_i - \mu)^2] + E[(\bar{x} - \mu)^2] - 2E[(x_i - \mu)(\bar{x} - \mu)] \right]$$

$$= \frac{1}{N} \sum_i \left[V[x_i] + V[\bar{x}] - 2E[(x_i - \mu)(\bar{x} - \mu)] \right]$$

$$= \frac{1}{N} \sum_i V[x_i] + \frac{1}{N} \sum_i V[\bar{x}] - \frac{2}{N} \sum_i E[(x_i - \mu)(\bar{x} - \mu)]$$

$$= \frac{1}{N} \sum_i V[x_i] + \frac{1}{N} \sum_i V[\bar{x}] - \frac{2}{N} \sum_i V[x_i]$$

$$= V[x] + V[\bar{x}] - \frac{2}{N} V[x]$$

$$= V[x] + \frac{1}{N} V[x] - \frac{2}{N} V[x]$$

$$= \left[1 - \frac{1}{N}\right] V[x]$$

a bit weird, but works
(eventually)

$$E[V_x] = \left[\frac{N-1}{N}\right] V[x]$$

$$V_x = \left[\frac{N-1}{N}\right] V[x]$$

$$\sum_i E[(x_i - \mu)(\bar{x} - \mu)]$$

$$\sum_i E[(x_i - \mu) \left(\frac{1}{N} \sum_j x_j - \mu \right)]$$

$$* \bar{x} = \frac{1}{N} \sum_i x_i$$

$$\sum_i \frac{1}{N} E[(x_i - \mu) (\sum_j x_j - N\mu)]$$

$$\sum_i \frac{1}{N} E[(x_i - \mu) (\sum_j (x_j - \mu))]$$

$$* \sum_j \mu = N\mu$$

$$\sum_i \frac{1}{N} E\left[\sum_j (x_i - \mu)(x_j - \mu)\right]$$

$$\frac{1}{N} \sum_i \sum_j E[(x_i - \mu)(x_j - \mu)]$$

IF $i \neq j$:

$$E[x_i - \mu] E[x_j - \mu] = (E[x_i] - \mu)(E[x_j] - \mu) = (\mu - \mu)(\mu - \mu) = 0$$

IF $i = j$:

$$E[x_i - \mu] E[x_i - \mu] = E[(x_i - \mu)^2] \neq 0$$

$$\frac{1}{N} \sum_i E[(x_i - \mu)^2]$$

$$\frac{1}{N} \sum_i V[x_i]$$

$$* E[(x - \mu)^2] = V[x]$$

$V_x = \frac{1}{N} \sum_i (x_i - \bar{x})^2$ IS BIASED ESTIMATOR FOR TRUE VARIANCE $V[x]$!!

$\left[\frac{N}{N-1}\right] \frac{1}{N} \sum_i (x_i - \bar{x})^2$ IS UNBIASED !!

$$= \frac{1}{N-1} \sum_i (x_i - \bar{x})^2$$

Sample Variance: Estimator Consistency

We have shown that:

$$\hat{V}[X] = V[x_N] = \frac{N-1}{N} V[X]$$

Consistency Check: Does $\hat{V}[X] \rightarrow V[X]$ as $N \rightarrow \infty$?

We can see that in the Asymptotic Limit ($N \rightarrow \infty$), $\frac{N-1}{N} \rightarrow 1$

The Sample Variance is a Consistent Estimator for the True Variance. ✓

Sample Variance: Estimator Bias

We have shown that:

$$\hat{V}[X] = V[x_N] = \frac{N-1}{N} V[X]$$

Bias Check: $b = E[\hat{V}[X]] - V[X]$

The bias is $b = \frac{N-1}{N} V[X] - V[X] = \frac{N-1-N}{N} V[X] = -\frac{1}{N} V[X]$

The Sample Variance is a Biased Estimator for the True Variance. ✖

We could use the Unbiased Sample Variance, $V[x_{N-1}] = \frac{1}{N-1} \sum_i (x_i - \bar{x})^2$, but bias is not the only factor in deciding which estimator is best...more next week.

The Sample Variance is a Biased Estimator

Does it make sense that the Sample Variance $V[x_N] = \frac{1}{N} \sum_i^N (x_i - \bar{x})^2$ is biased?

$b = E[\hat{\theta}] - \theta = -\frac{1}{N}\theta$: the bias is **negative**, because $E[\hat{\theta}]$ is **smaller** than the true variance θ .*

We can make sense of this if we consider the data used in each case. **The Sample Variance uses a limited number of measurements, N . These measurements determine the sample mean, $\bar{x}$, and are therefore clustered around the sample mean.**

The value of the True Mean is formed by all possible measurements, so it makes sense that the true variance will be larger than the sample variance. The larger N gets, the closer we get to 'all possible measurements', so the bias reduces.

*Using $E[\hat{\theta}] \equiv E[\hat{V}[X]] \equiv E[V[x_N]]$ and $\theta \equiv V[X]$ for neatness.

A Reminder of the Likelihood

When we calculate probabilities, we assume a known parameter, eg λ or p . The general symbol for any such parameter, or set of them, is θ .

Sometimes we may not know the parameter value, and wish to Estimate it based on the distribution of our **data**. We can do this by trying out a bunch of different parameter values and seeing which gives us the 'closest match' in shape to the distribution of our **data**. The parameter that results in the closest match is the one with the highest Likelihood, $\mathcal{L}_\theta$.

The Likelihood $\mathcal{L}_\theta$ of a parameter value θ being the correct one, **given the data**, is:

$$\mathcal{L}_\theta = \prod_i^N f_Y(y_i; \theta)$$

A Reminder of the Likelihood

$$\mathcal{L}_\theta = \prod_i^N f_Y(y_i; \theta)$$

The Likelihood is the product of the PDF f_Y **evaluated at each data measurement y_i** . The **product is over the N data measurements**. θ is the parameter value (or set of) that we are trying out. The parameter value that best describes the data is the one with the highest $\mathcal{L}_\theta$. We recalculate $\mathcal{L}_\theta$ for each value of θ we want to try out using the above formula.

Wait, didn't you say that the PDF evaluated at a given value is zero?!

Yes I did! The difference here is that each data measurement represents a range of values, rather than a single value (we can't define a single value from a continuous set of real numbers). Measurements are always discrete, even when the RV we are measuring is continuous. So, $f_Y(y_i; \theta) \neq 0$.

A Reminder of the Likelihood

In preparing these slides I went back to the week 3 Functions lecture to steal some L^AT_EX, and found an error* in my definition of the Likelihood. Sorry!

This is an important definition that I wrote down completely backwards.*

The Normalised Likelihood is the Probability of θ , given the data. ✖

I have corrected this on the slides:

The Normalised Likelihood is the Probability of the data, given θ .



Let's go back to Bayes-ics (sorry) to check we understand.

*A frequentist would say that what I wrote was not in error, because the two statements mean the same thing. But this is not the case for the Bayesian approach, which is what I was attempting to convey. So it was an error.

A Reminder of Bayes' Theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

All of the Ps in here are **valid** PDFs (or PMFs): they have integral (or sum) equal to 1. ✓

A Reminder of Bayes' Theorem

Let $A \rightarrow \theta$: the parameter values, and $B \rightarrow y$: the data.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$



$$P(\theta|y) = \frac{P(y|\theta)P(\theta)}{P(y)}$$

Are all of the Ps still **valid**?

A Reminder of Bayes' Theorem

Let $A \rightarrow \theta$: the parameter values, and $B \rightarrow y$: the data.

$$P(\theta|y) = \frac{P(y|\theta)P(\theta)}{P(y)}$$

$P(\theta)$ is the Total Probability that the correct value of our parameter is ... anything! ✓

A Reminder of Bayes' Theorem

Let $A \rightarrow \theta$: the parameter values, and $B \rightarrow y$: the data.

$$P(\theta|y) = \frac{P(y|\theta)P(\theta)}{P(y)}$$

$P(y)$ is the Total Probability of our data having any value. ✓

A Reminder of Bayes' Theorem

Let $A \rightarrow \theta$: the parameter values, and $B \rightarrow y$: the data.

$$P(\theta|y) = \frac{P(y|\theta)P(\theta)}{P(y)}$$

$P(y|\theta)$ is the probability of observing all the data, given any parameter value is correct. ✓

A Reminder of Bayes' Theorem

Let $A \rightarrow \theta$: the parameter values, and $B \rightarrow y$: the data.

$$P(\theta|y) = \frac{P(y|\theta)P(\theta)}{P(y)}$$

$P(\theta|y)$ is the probability of any parameter value being correct, given all the data. ✓

A Reminder of Bayes'

The next step is to consider how we can use this to talk about the probability of a specific parameter θ_i being the correct one.

$$P(\theta|y) = \frac{P(y|\theta)P(\theta)}{P(y)}$$

Let's see what happens...

A Reminder of Bayes'

The next step is to consider how we can use this to talk about the probability of a specific parameter θ_i being the correct one.

$$P(\theta_i|y) = \frac{P(y|\theta_i)P(\theta_i)}{P(y)}$$

We have broken Bayes! The Ps are no longer (necessarily) valid! They have some meaning, but they are not (necessarily) Probabilities.

A Reminder of Bayes'

The next step is to consider how we can use this to talk about the probability of a specific parameter θ_i being the correct one.

$$P(\theta_i|y) = \frac{P(y|\theta_i)P(\theta_i)}{P(y)}$$

We must fix this! Let's work with our one remaining valid P, $P(y)$.

A Reminder of Bayes'

$P(y) = \int P(y, \theta) d\theta$ is a Marginal P of the Joint P $P(y, \theta)$.

Recall $f_Y(y) = \int_{-\infty}^{\infty} f_{YZ}(y, z) dz$ - we 'integrate out' the other RV.

$$P(\theta_i|y) = \frac{P(y|\theta_i)P(\theta_i)}{\int P(y, \theta) d\theta}$$

A Reminder of Bayes'

$P(y, \theta) = P(y|\theta) P(\theta)$: the Joint P can be written as the Conditional P times the Marginal P.

Recall $P(A \cap B) = P(A|B)P(B)$.

$$P(\theta_i|y) = \frac{P(y|\theta_i)P(\theta_i)}{\int P(y|\theta)P(\theta)d\theta}$$

A Reminder of Bayes'

With the new expression in the denominator $P(y) = \int P(y|\theta)P(\theta)d\theta$ we can clearly show that **the LHS is** a valid probability:

$$\int P(\theta_i|y)d\theta = \int \frac{P(y|\theta_i)P(\theta_i)}{\int P(y|\theta_i)P(\theta_i)d\theta}d\theta = \frac{\int P(y|\theta_i)P(\theta_i)d\theta}{\int P(y|\theta_i)P(\theta_i)d\theta} = 1$$

This is important, because it means that the Posterior probability $P(\theta_i|y)$ for a specific parameter θ_i , given the data, is a valid PDF.

A Reminder of Bayes'

$P(\theta_i|y)$, the Posterior probability distribution for the correct parameter being θ_i , given the data y , is a valid PDF. ✓

The so-called Ps on the RHS are not (necessarily) valid PDFs.

$$P(\theta_i|y) = \frac{P(y|\theta_i)P(\theta_i)}{\int P(y|\theta_i)P(\theta_i)d\theta}$$

Let's fix this, sharpish.

A Reminder of Bayes'

$P(\theta_i)$ is the **Prior** of the specific parameter θ_i being the correct one. We gave the Prior a special symbol $\pi(\theta_i)$ - we are going to get confused if we keep the P there, because it is not necessarily a valid PDF. ✖

$$P(\theta_i|y) = \frac{P(y|\theta_i)P(\theta_i)}{\int P(y|\theta_i)P(\theta_i)d\theta}$$

A Reminder of Bayes'

$\pi(\theta_i)$ is the **Prior** of the specific parameter θ_i being the correct one. We gave the Prior a special symbol $\pi(\theta_i)$ because it is not necessarily a valid PDF.

That's better.

$$P(\theta_i|y) = \frac{P(y|\theta_i)\pi(\theta_i)}{\int P(y|\theta_i)\pi(\theta_i)d\theta}$$

A Reminder of Bayes'

$P(y|\theta_i)$ is the Likelihood of observing the data given that a specific parameter θ_i is the correct one. We gave this Likelihood a special symbol $\mathcal{L}_{\hat{\theta}}$. On its own, it is not necessarily a valid PDF.✖

$$P(\theta_i|y) = \frac{P(y|\theta_i)\pi(\theta_i)}{\int P(y|\theta_i)\pi(\theta_i)d\theta}$$

A Reminder of Bayes'

$\mathcal{L}_\theta$ is the Likelihood of observing the data given that a specific parameter θ_i is the correct one. On its own, it is not a valid PDF. That's better.

$$P(\theta_i|y) = \frac{\mathcal{L}_\theta \pi(\theta_i)}{\int \mathcal{L}_\theta \pi(\theta_i) d\theta}$$

A Reminder of Bayes'

Now we are happy. ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

$$P(\theta_i|y) = \frac{\mathcal{L}_\theta \pi(\theta_i)}{\int \mathcal{L}_\theta \pi(\theta_i) d\theta}$$

Later on we will see that the denominator is just a normalising factor, and the Posterior is $\propto \mathcal{L}_\theta \times \pi(\theta)$. For the Frequentist, the Posterior is just the (normalised) Likelihood.

The Maximum Likelihood: Intro

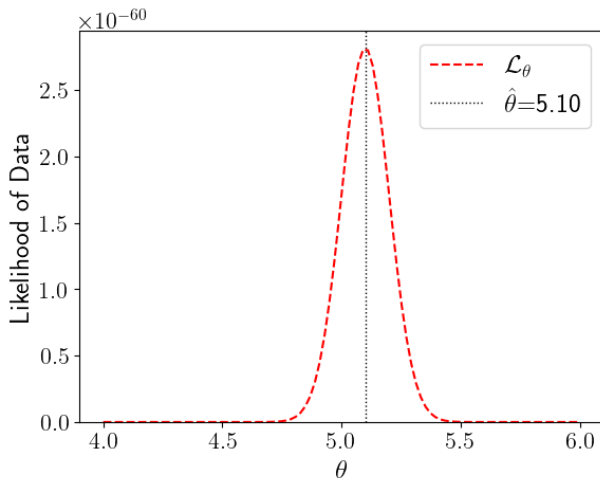
$\mathcal{L}_\theta$ is the Likelihood of observing the data given that a specific parameter θ_i is the correct one.

The value θ_i that **Maximises the Likelihood** is, by definition, the best Estimate $\hat{\theta}$ for the true parameter θ .

$$\mathcal{L}_\theta = \prod_i^N f_Y(y_i; \theta)$$

To find the maximum of a function, we differentiate it, and set the result to zero. This is because **the first derivative of a function at a given point is its Gradient**.

The Maximum Likelihood



The Maximum Likelihood Estimator (MLE)

The value θ_i that **Maximises the Likelihood** is, by definition, the best Estimate $\hat{\theta}$ for the true parameter θ .

$$\mathcal{L}_{\theta} = \prod_i^N f_Y(y_i; \theta)$$

- We can use the MLE when we know the form of the PDF (f_Y) but not its parameter values.
- The MLE method works by evaluating $\mathcal{L}_{\theta}$ for different θ_i to determine the value that maximises $\mathcal{L}_{\theta}$.
- We shall see later on that maximising the Likelihood is equivalent to **Minimising The Residuals**.

Residuals

- A dataset $y = \{y_i, \dots, y_N\}$ contains the measured values of a RV Y from some underlying PDF $f_Y(y; \theta)$.
- The **Residual** for the i^{th} measurement is $y_i - \mu$: a measure of how far the measurement y_i deviates from the True Mean $\mu \equiv E[Y]$.
- The **Standardised Residual** for the i^{th} measurement is $\frac{y_i - \mu}{\sigma_i} = \frac{y_i - \mu}{y_i - \bar{y}}$.

To calculate the **Residual**, $y_i - \mu$, we must know (or estimate) the underlying distribution for our data, as μ is not (necessarily) a statistic of the data but a parameter of the true distribution. However, if we estimate μ from the data, then the residual is a statistic of the data.

The denominator is the **Measurement Uncertainty** $\sigma_i = y_i - \bar{y}$. This is a statistic, purely data-based, making no assumptions as to the underlying distribution.

Chi Squared

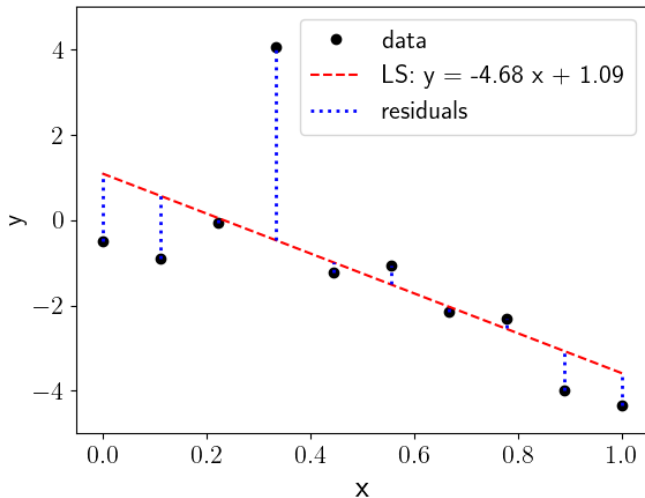
Pearson's Chi Squared, χ^2 , is the sum of the squared standardised residuals:

$$\chi^2 = \sum_i^N \frac{(y_i - \mu)^2}{V[y_i]}$$

We will become familiar with this. For now, notice that the smaller χ^2 is, the closer (on average) our measurements are to the expectation.

Out in the world you will see the standardised residual written as $\frac{\text{obs} - \text{exp}}{\sqrt{\text{exp}}}$, where obs and exp are counts (numbers). We will see later that this is the form used for Binned Data (histograms), because in that case we estimate the statistical uncertainty on the number of entries in a given bin as $\sigma_{bi} = \sqrt{N}$, where N is the number of expected entries in the bin. This is known as Poisson statistics, or 'counting statistics'.

Least Squares

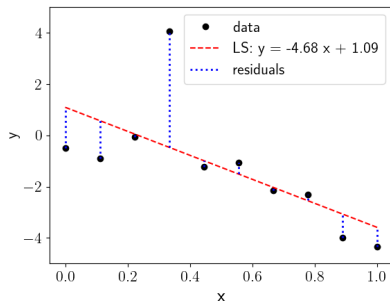


Least Squares

When I hear the term Least Squares I think of sitting in an undergrad laboratory trying to fit a line to some data points such that the deviations between the data points and the line are as small as possible.

This means **Minimising The Residuals**.

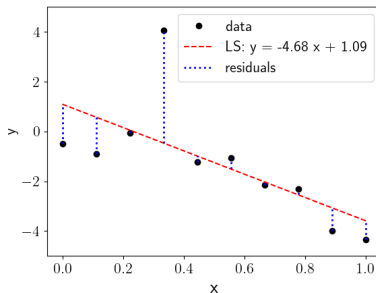
I think one reason why Least Squares is used so prolifically is that it is understandable visually. Do not underestimate the power of a good plot.



Least Squares

The method is called Least Squares because it chooses the (best fit) line for which the sum of the squared residuals (Squares) is minimised (Least).

We can fit any line or curve (any functional form) to data points - we don't need to know what the functional form is in advance, because we can try everything! It does help to have an educated guess to start with, though.



LSE and MLE: Point Estimators for Parameters

Both Least Squares and Max Likelihood are Point Estimators for finding the parameter Estimates that Maximise $\mathcal{L}_\theta$ by Minimising the Residuals.

- Least Squares looks pretty easy and intuitive, and we can see the residuals directly, but it's not yet clear how they relate to the Likelihood...
- Maximising the Likelihood just looks like math, and although I said it is related to minimising the residuals, I have not yet given you any idea of how (other than a slide last week where I showed the log of a Normal PDF...)

To use LSE we need to have a really good grasp of our uncertainties (error bars) because the larger the error bars, the more possibilities there are for curves that will fit the data. We can do basic LSE with just a few data points, hence its popularity in undergraduate labs.

To use MLE we need to know the underlying PDF, and we need a pretty good amount of data for MLE to work its magic.

The Log Likelihood

Logarithms are very handy. Recall $\ln(abc) = \ln a + \ln b + \ln c$

The Log of a product is the sum of logs:

$$\ln \mathcal{L} = \ln \left(\prod_i^N f_X(x_i; \theta_i) \right) = \sum_i^N \ln f_X(x_i; \theta)$$

The sum is over the PDF evaluated at each of the N data measurements x_i .

It is generally much easier to work with the log likelihood than the likelihood.

The Negative Log Likelihood

The Log Likelihood $\ln \mathcal{L} = \sum_i^N \ln f_X(x_i; \theta)$ is always negative, because $\ln(a) < 0$ for $a < 1$, and $f_X(x_i; \theta)$ must always be less than 1.

Because the Log Likelihood is negative and **we like positive numbers***, we pop a minus sign on the front and use the Negative Log Likelihood for our calculations.

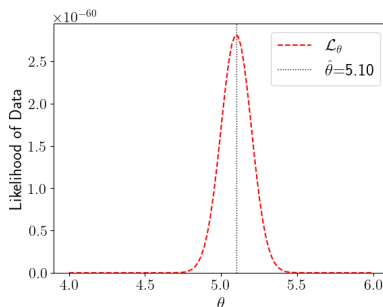
$$-\ln \mathcal{L} = -\sum_i^N \ln f_X(x_i; \theta)$$

Maximising the Likelihood is equivalent to Minimising the Negative Log Likelihood.

***This is me being silly. Finding the Point Estimate $\hat{\theta}$ is easiest using `scipy.optimize` methods, and optimisers are minimisers rather than maximisers, by convention.**

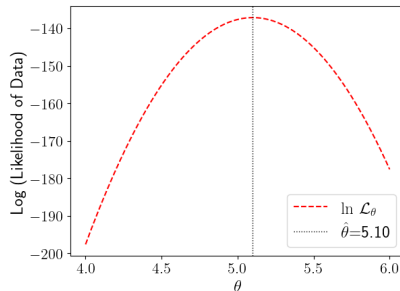
Likelihood, Log Likelihood, and Negative Log Likelihood

Likelihood $\mathcal{L}_\theta$: Maximum at $\hat{\theta}$



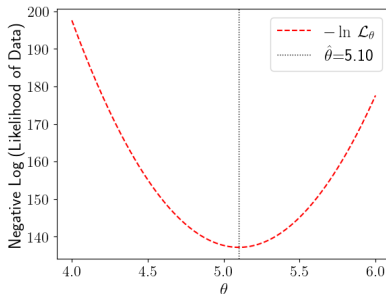
Likelihood, Log Likelihood, and Negative Log Likelihood

Log Likelihood $\ln \mathcal{L}_\theta$: Maximum at $\hat{\theta}$ and we can see the tails!



Likelihood, Log Likelihood, and Negative Log Likelihood

Negative Log Likelihood $-\ln \mathcal{L}_\theta$: Minimum at $\hat{\theta}$ and we can see the tails!



All the same information that was in the Likelihood $\mathcal{L}_\theta$ is in the Negative Log Likelihood $-\ln \mathcal{L}_\theta$, it is just nicer to work with $-\ln \mathcal{L}_\theta$.

Thanks For Your Time

Fin.